Fourier Neural Operator 2D(FNO2D) for 2D Topography

I. What are Fourier Neural Operators ?

Neural Operators: *learn mappings between input functions and output functions*

$$G : (a(x), f(x), g(x)) \mapsto u(x)$$ *(gemini)*

Fourier Neural Operators: *learning operators via global spectral convolutions parameterized in the Fourier (spectral) domain*

$$Q \circ \sigma(W_L + K_L) \circ \cdots \circ \sigma(W_1 + K_1) \circ P.$$ [1]

A Simple Single-Layer FNO Structure

Suppose we have discrete data from an input function and output function

$G: f(x) \longmapsto g(x)$

(sample size)

input: $(x = -1, -0.33, 0.33, 1)$ } 4 points

output: $(x^2 = 1, 0.11, 0.11, 1)$

here we know $f(x) = x$, $g(x) = x^2$

$$\underbrace{\overset{3.}{Q} \circ \overset{2.}{\sigma(W_1 + K_1)} \circ \overset{1.}{P}}$$

1. Lifting (P) $f(x)$, $x \in \mathbb{R}$ $\rightarrow$ $P(f)(x) = \begin{bmatrix} \phi_1(x_1) \\ \vdots \\ \phi_d(x_1) \end{bmatrix} \in \mathbb{R}^d$, here assume $d = 2$

$\phi_1 = 2x + 1$ $\phi_2 = 3x - 1$

take 1 sample as an example

batch 1 $\vec{x} = \begin{bmatrix} -1 \\ -0.33 \\ 0.33 \\ 1 \end{bmatrix} \xrightarrow{\vec{x}\underline{A} + \underline{B}} \begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) \\ \phi_1(x_2) & \phi_2(x_2) \\ \phi_1(x_3) & \phi_2(x_3) \\ \phi_1(x_4) & \phi_2(x_4) \end{bmatrix}$ Linear lifting

2. Nonlinear (σ) projection of Fourier (K) Kernel and Pointwise (W) Kernel

σ nonlinear projection

here we assume $\sigma = \max(0, x)$

Discrete Fourier Transform (DFT)

$K = \mathcal{F}^{-1}[\hat{W}(k)\mathcal{F}(\vec{x})]$

IDFT

$W = \vec{x}\underline{C} + \underline{D}$

3. Projection (Q)

project back to target function x^2

$A^{4\times 2} Q^{2\times 1} = T^{4\times 1}$ $\begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$

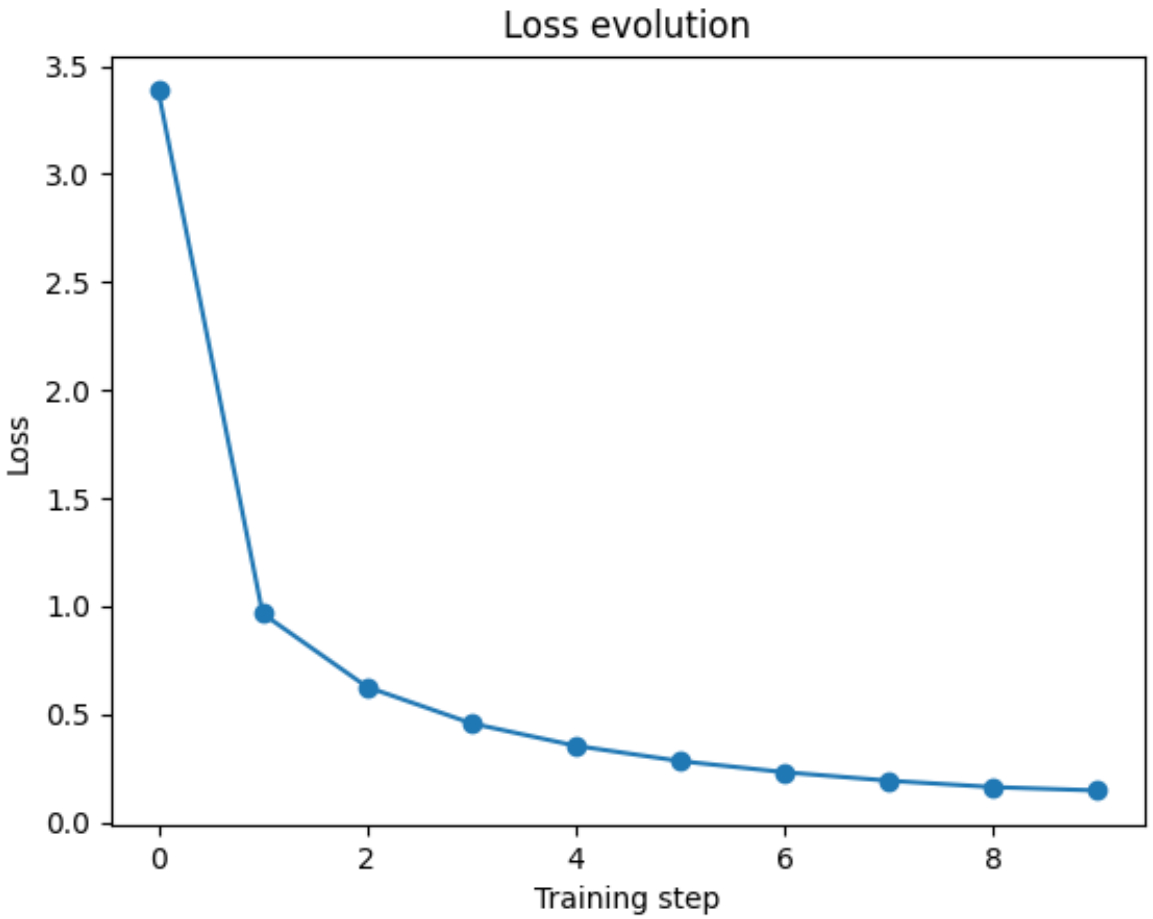
$L = \frac{1}{N}\sum_{n=0}^{3}(\text{predicted} - \text{true})^2$

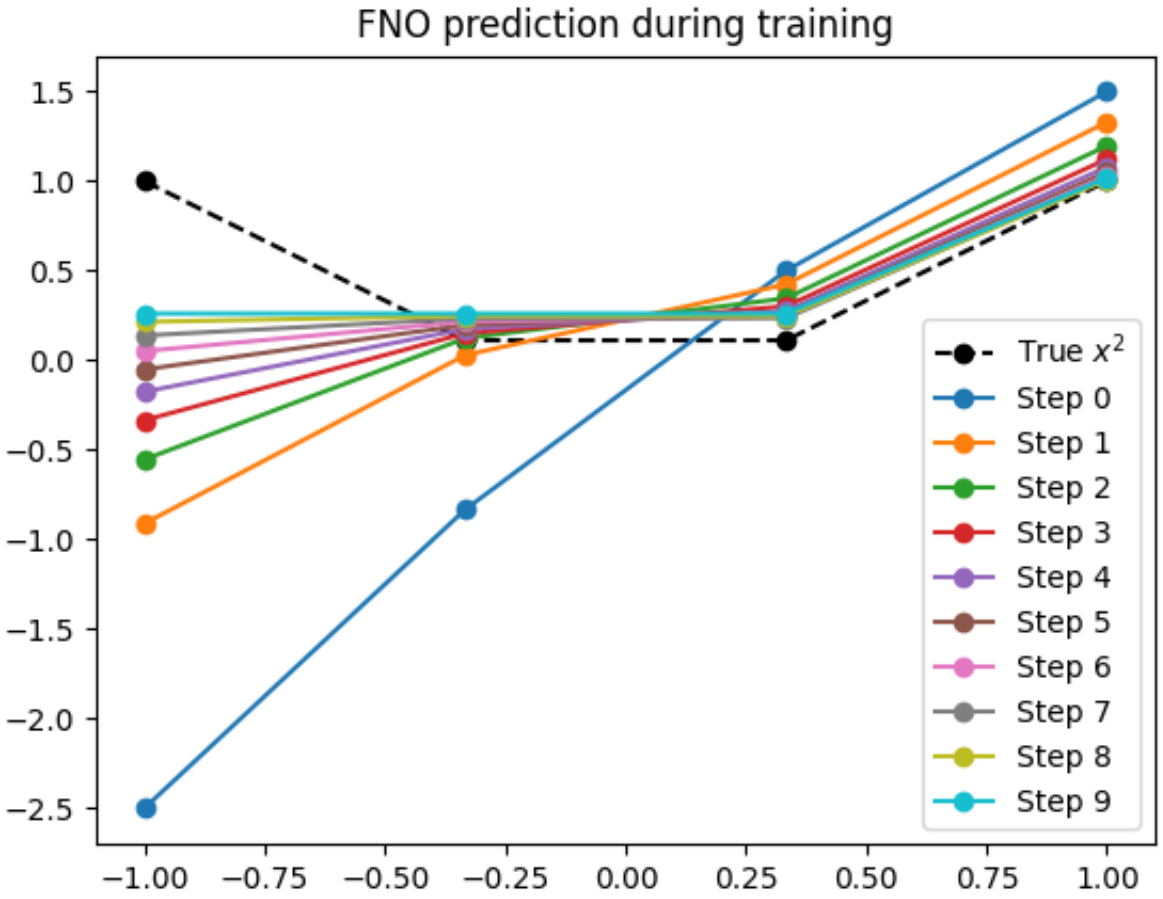
Then the gradient of each learnable parameter is calculated $\frac{\partial L}{\partial P}$, $\frac{\partial L}{\partial \hat{W}(k)}$, $\frac{\partial L}{\partial W}$, $\frac{\partial L}{\partial Q}$ and the parameters are updated by $w_0 - \eta \frac{\partial L}{\partial w}$ . . .

learning rate

Processes 1, 2, and 3 are performed iteratively until the required accuracy is achieved

Graphical Illustration

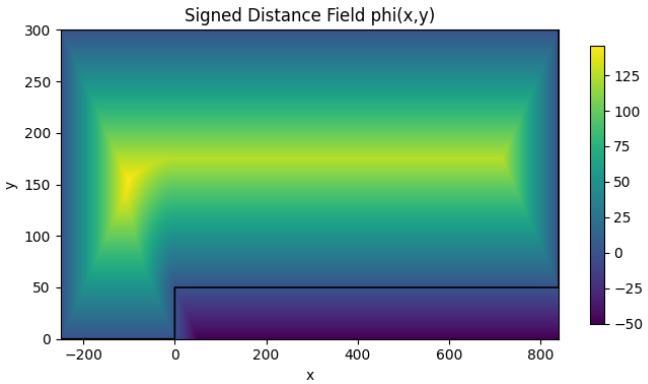


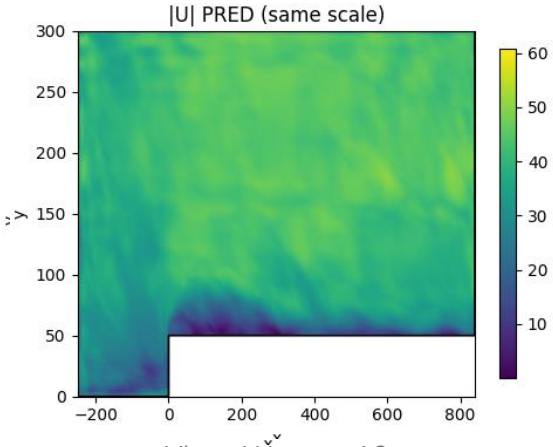


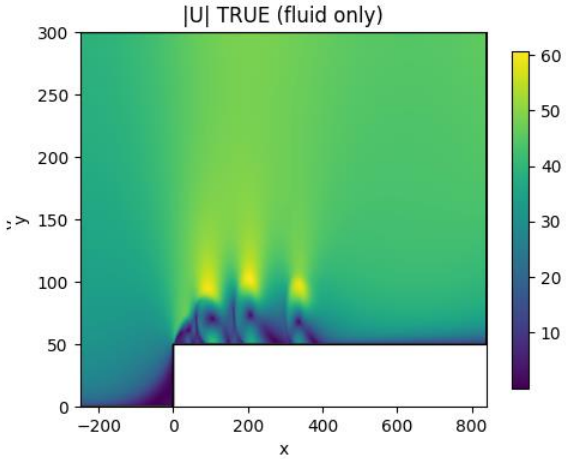
II. What Is Done?

A FNO Network with a Physics-Informed Loss Function

- Intention: *To investigate whether Fourier Neural Operators (FNOs) can be used to build an efficient model that achieves acceptable accuracy for solving the 2D Navier–Stokes equations in wind engineering applications*
- Data: *362 cases for training*
- Structure: *The topology function is used as the input, while* u, k, *and* ε *are treated as output functions*
- Observation: *The model is expected to perform better with increased data availability*

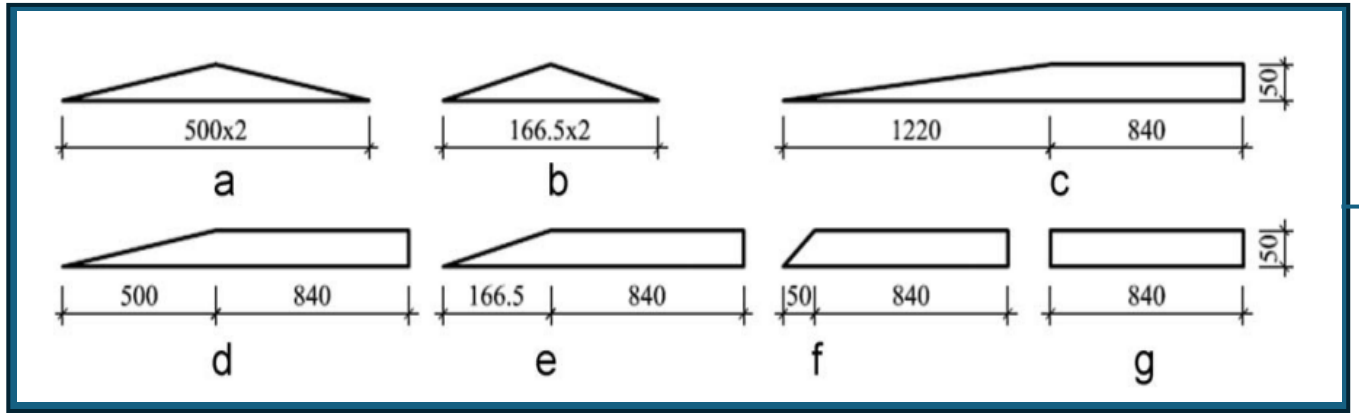




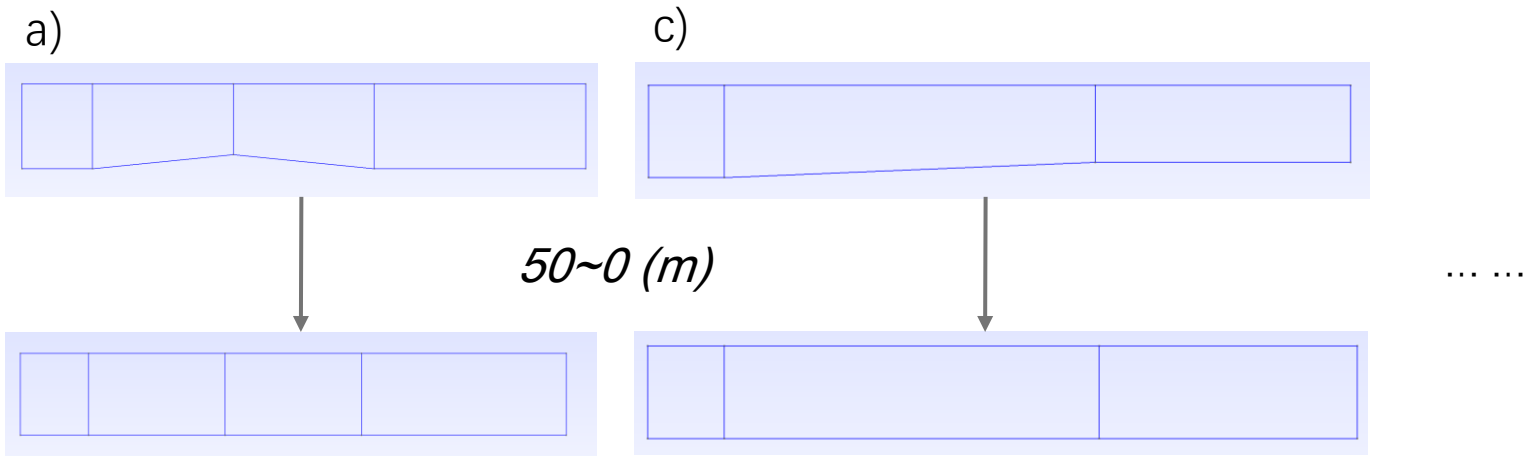


II.1 Data

1.Typical cases are taken from https://doi.org/10.1016/j.jweia.2009.06.013



For each a~g, 50 RANS cases were generated using OpenFOAM by varying the height of the top edge of the windward face from 50 to 0.



Data

Fixed velocity input

```
boundaryField
{
    outlet
    {
        type            inletOutlet;
        inletValue      uniform (0 0 0);
        value           $internalField;
    }

    inlet
    {
        type            atmBoundaryLayerInletVelocity;
        #include        "include/ABLConditions"
    }

    ground
    {
        type            uniformFixedValue;
        uniformValue    (0 0 0);
        value           uniform (0 0 0);
    }

    #include "include/sideAndTopPatches"
}
```

```
Uref                24.0;
Zref                10;
zDir                (0 1 0);
flowDir             (1 0 0);
z0                  uniform 0.05;
zGround             uniform 0;
```

Computational domain

```
(
    inlet
    {
        type            patch;
        nFaces          150;
        startFace       298850;
    }
    outlet
    {
        type            patch;
        nFaces          150;
        startFace       299000;
    }
    ground
    {
        type            wall;
        inGroups        List<word> 1(wall);
        nFaces          1000;
        startFace       299150;
    }
    top
    {
        type            patch;
        nFaces          1000;
        startFace       300150;
    }
    sides
    {
        type            empty;
        inGroups        List<word> 1(empty);
        nFaces          300000;
        startFace       301150;
    }
)
```

Data

Typical Convergence

```
smoothSolver:  Solving for Ux, Initial residual = 8.5881e-07, Final residual = 8.5881e-07, No Iterations 0
smoothSolver:  Solving for Uy, Initial residual = 2.69172e-05, Final residual = 3.36205e-07, No Iterations 1
GAMG:  Solving for p, Initial residual = 0.00180536, Final residual = 8.63766e-06, No Iterations 1
GAMG:  Solving for p, Initial residual = 8.63035e-06, Final residual = 3.36776e-07, No Iterations 1
time step continuity errors : sum local = 1.6142e-10, global = -1.89181e-11, cumulative = -9.77407e-07
smoothSolver:  Solving for epsilon, Initial residual = 0.0621769, Final residual = 0.000452238, No Iterations 1
smoothSolver:  Solving for k, Initial residual = 9.97915e-07, Final residual = 9.97915e-07, No Iterations 0
```

```
smoothSolver:  Solving for Ux, Initial residual = 0.00356686, Final residual = 8.01942e-05, No Iterations 2
smoothSolver:  Solving for Uy, Initial residual = 0.0197854, Final residual = 0.000437781, No Iterations 2
GAMG:  Solving for p, Initial residual = 0.209774, Final residual = 0.00782456, No Iterations 2
GAMG:  Solving for p, Initial residual = 0.00931624, Final residual = 0.000297024, No Iterations 5
time step continuity errors : sum local = 3.30751e-06, global = -4.47363e-09, cumulative = -1.52705e-06
smoothSolver:  Solving for epsilon, Initial residual = 0.0589818, Final residual = 0.00074323, No Iterations 1
smoothSolver:  Solving for k, Initial residual = 0.00339056, Final residual = 5.84789e-05, No Iterations 2
```

```
smoothSolver:  Solving for Ux, Initial residual = 0.00392647, Final residual = 8.19932e-05, No Iterations 2
smoothSolver:  Solving for Uy, Initial residual = 0.015111, Final residual = 0.000270861, No Iterations 2
GAMG:  Solving for p, Initial residual = 0.169575, Final residual = 0.00578565, No Iterations 2
GAMG:  Solving for p, Initial residual = 0.00547511, Final residual = 0.000254045, No Iterations 5
time step continuity errors : sum local = 3.8447e-06, global = 3.52448e-08, cumulative = 1.89585e-05
smoothSolver:  Solving for epsilon, Initial residual = 0.0370196, Final residual = 0.000859197, No Iterations 1
smoothSolver:  Solving for k, Initial residual = 0.00403188, Final residual = 0.000379103, No Iterations 1
```

II.2 Structure

$$G: (\phi, X, Y) \longrightarrow (U_x, U_y, k, \varepsilon)$$

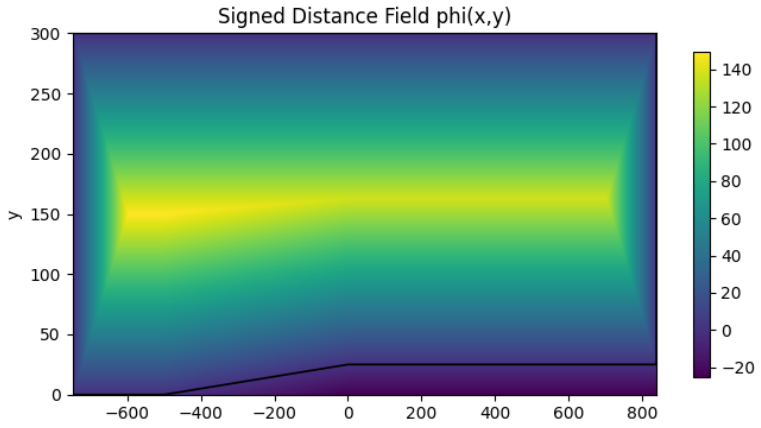
a. Input

$\mathbf{x} \in \mathbb{R}^{N_x \times N_y \times 3}$ *(GPT)*

signed distance function

$$\phi(x,y) = \begin{cases} +d((x,y), \partial\Omega), & (x,y) \in \Omega \\ -d((x,y), \partial\Omega), & (x,y) \notin \Omega \end{cases}$$

$$d(x,y) = \min_{(x_b,y_b)\in\partial\Omega} \sqrt{(x-x_b)^2 + (y-y_b)^2}$$



- Φ>0 inside (fluid)
- Φ<0 outside (wall)
- Φ=0 boundary

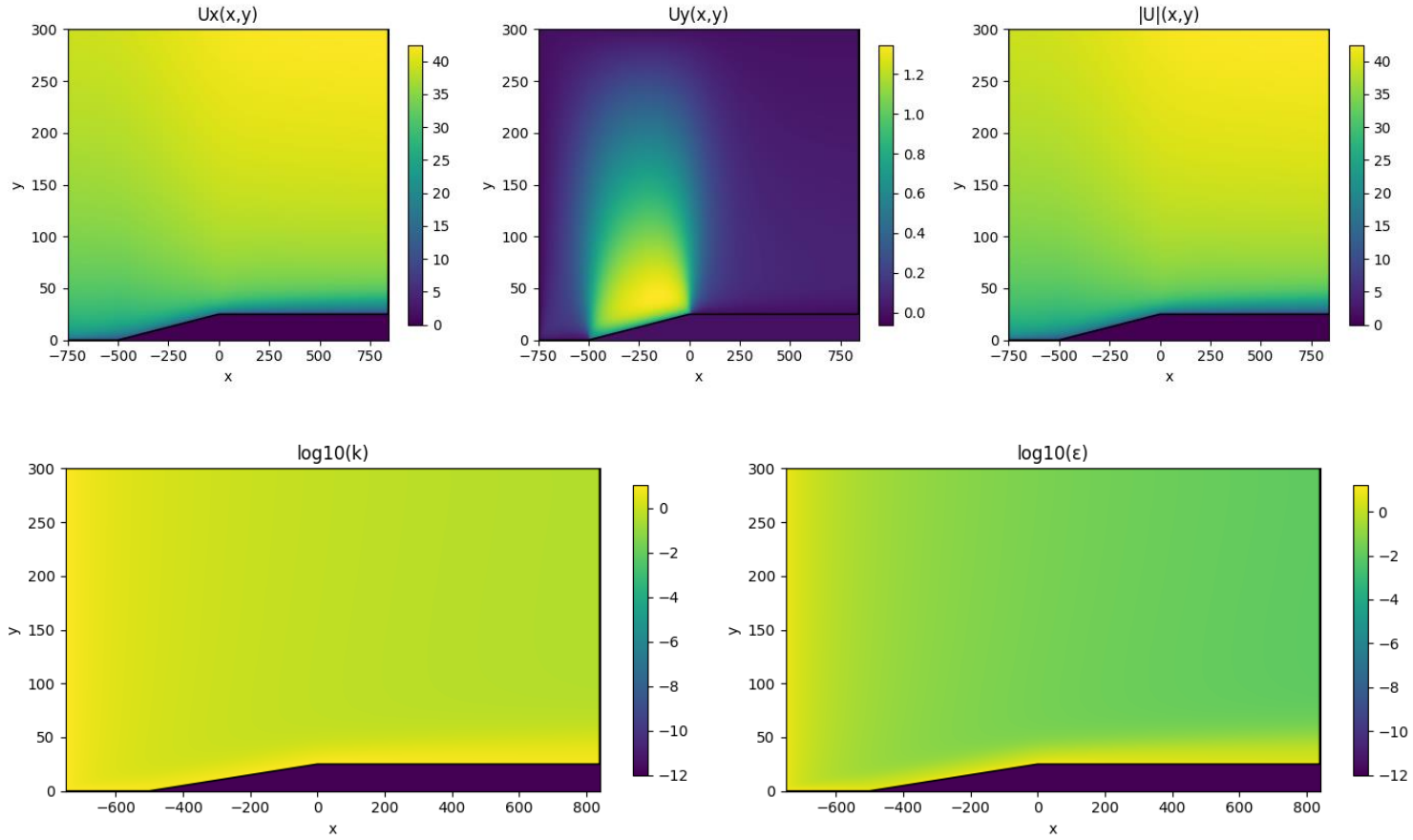
X,Y are normalized to [-1,1]

$$X_{norm} = 2\frac{X - X_{min}}{X_{max} - X_{min}} - 1$$

- *A uniform resolution of **Nx=1024×Ny=512** is used for the signed distance function, onto which the OpenFOAM solver results are subsequently mapped using linear interpolation.*

Structure

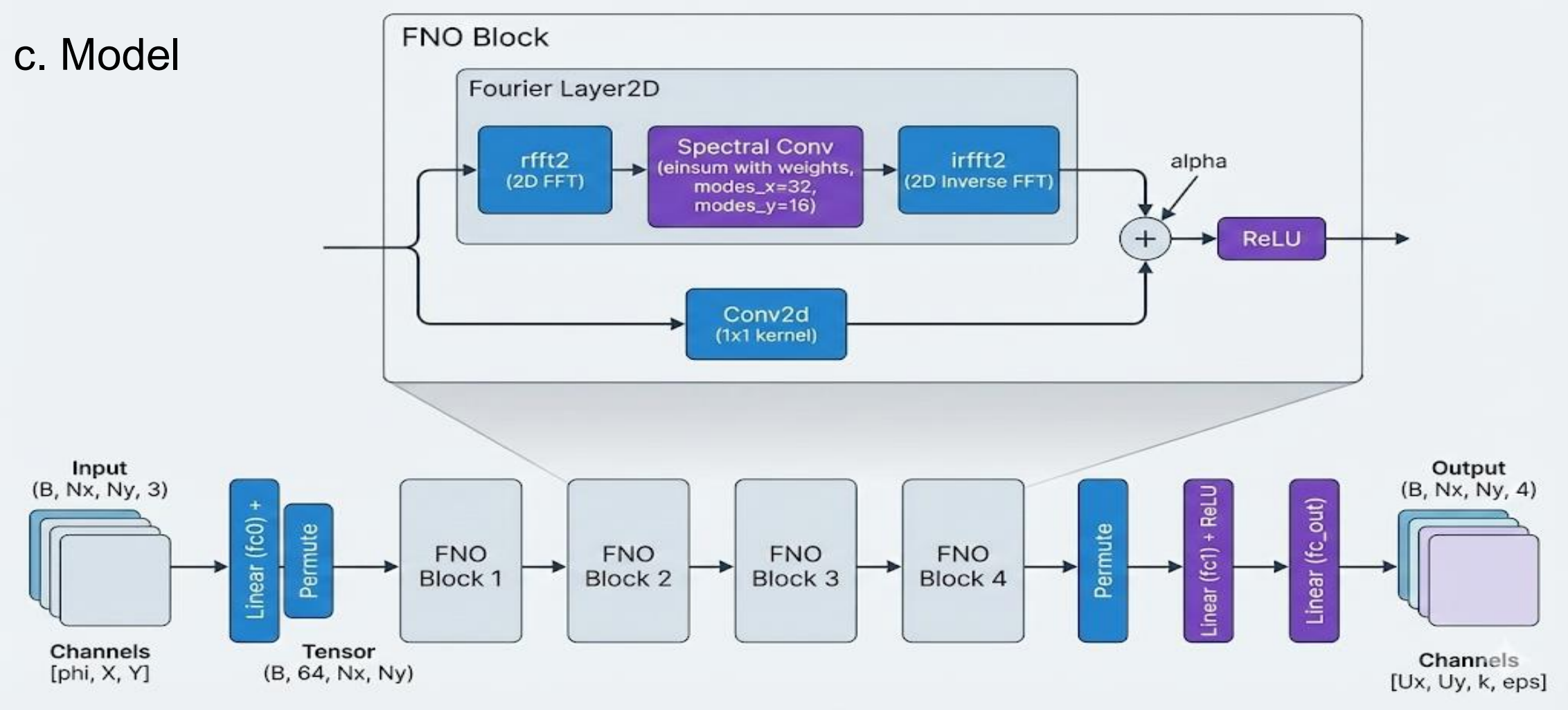
b. Target output $\mathbf{y} \in \mathbb{R}^{N_x \times N_y \times 4}$ *(GPT)*



- *linearly interpolated on to the 1024×512 grid*

Structure

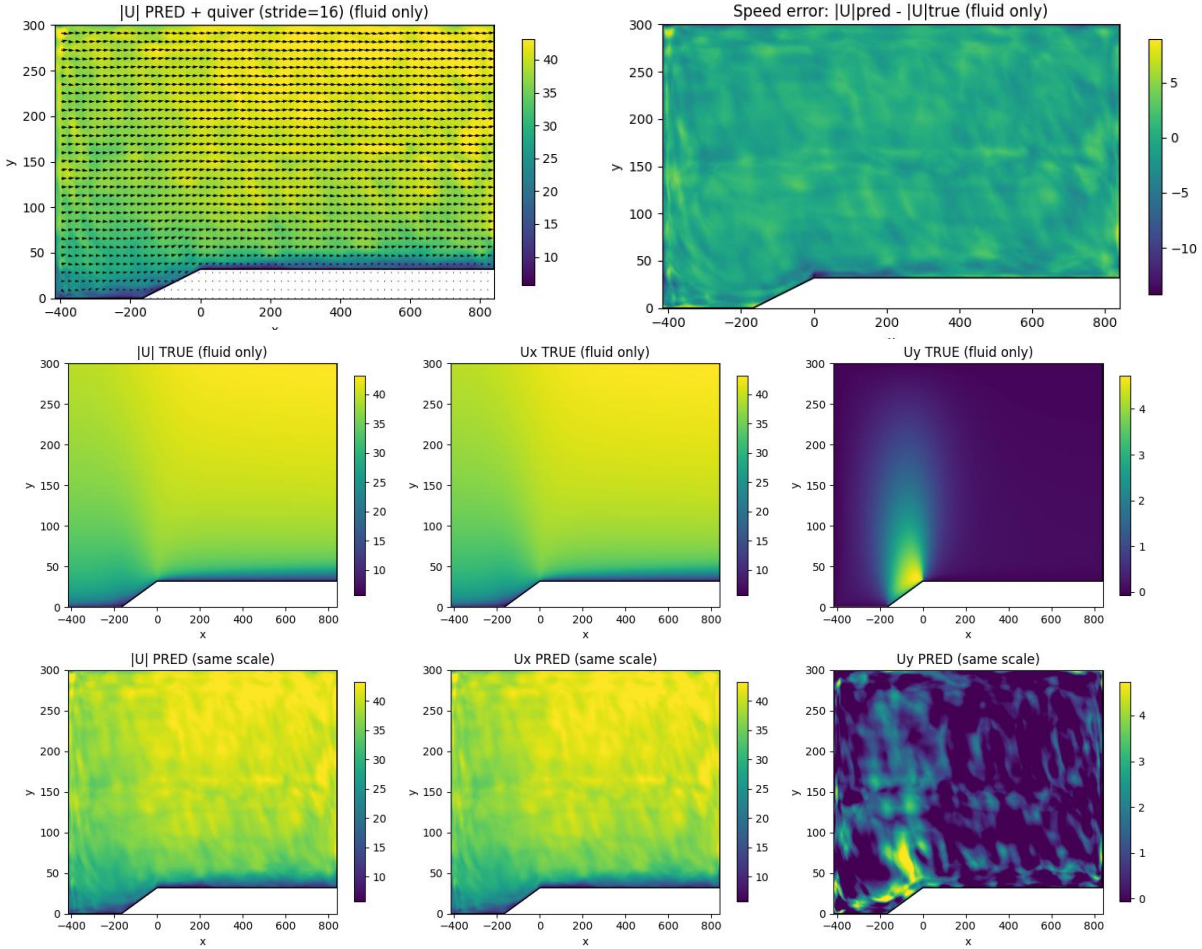
c. Model



(gemini)

II.3 Observation

a. Difference between predicted and true values:



- What might causes smooth-but-wrong oscillations
  - small number of Fourier modes ($N_x = 32$, $N_y = 16$ (is insufficient for a Navier–Stokes problem characterized by highly turbulent flow.
  - not penalized for non-physical oscillations only $\nabla \cdot \mathbf{u} \approx 0$ is required in the model *(GPT)*

- No sharp memorization artifacts
- Errors are global, not localized

b. Next Step

- Larger training data
- Needs better computation power
- refine the physical loss in the model to let it learn the actual physical behavior

c. Advantages

- Its fast <20s after training
- solution is queried from the learned function rather than computed on a fixed grid (no mesh needed after training)

d. limitations

- Currently, it learns from data generated by a numerical solver rather than from actual physical behavior
- Currently, the velocity input is fixed for feasibility analysis

Key Reference

[1] https://doi.org/10.48550/arXiv.2512.01421